UNIVERSIDAD NACIONAL DE UCAYALI

FACULTAD DE INGENIERÍA DE SISTEMAS E INGENIERÍA CIVIL

ESCUELA DE PROFESIONAL DE INGENIERÍA DE SISTEMAS

SISTEMA CONTROL DE VERSIONES

PROGRAMACIÓN ORIENTADA A OBJETOS II

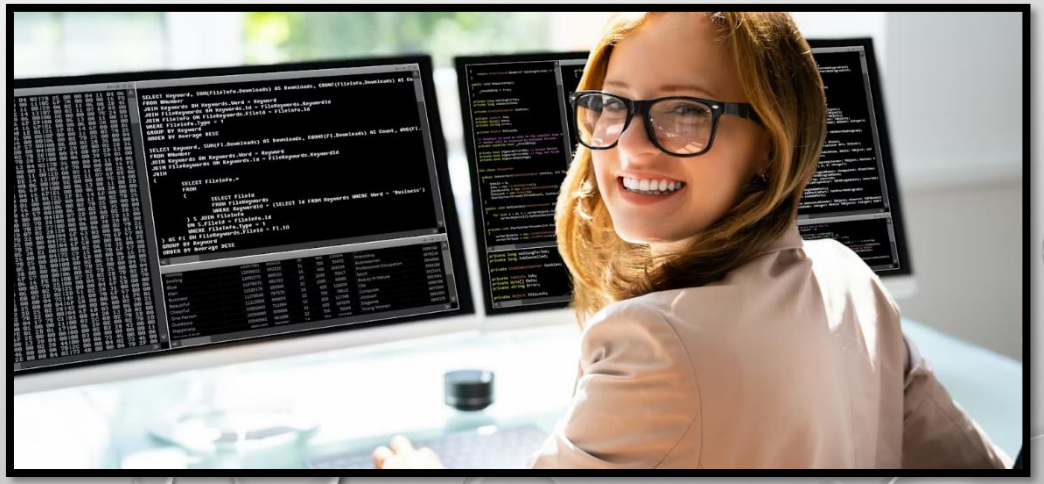
Docente : Ing. M.Sc. Edson Greig Cáceres Cárdenas

Pucallpa – Perú

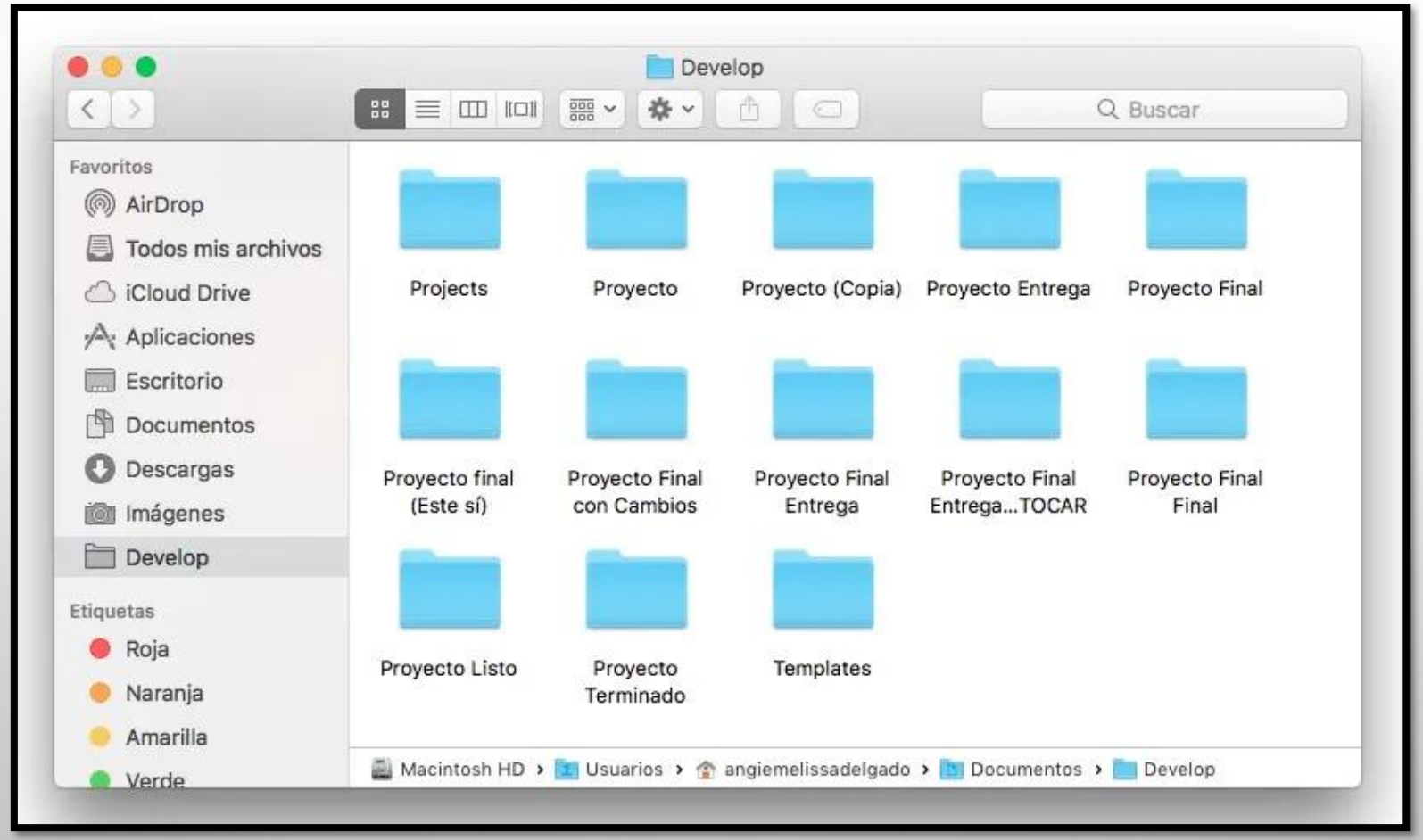
2026

SISTEMA CONTROL DE VERSIONES

La vida de un desarrollador de software (y el orden de nuestros sistemas de archivos) se encuentra dividido por un gran hecho: el antes y el después de conocer los sistemas de control de versiones. Y es que debemos admitirlo, nuestra vida cambia cuando conocemos y empezamos a utilizar los sistemas de control de versiones. Pero vamos a empezar hablando de esta etapa oscura: el antes.

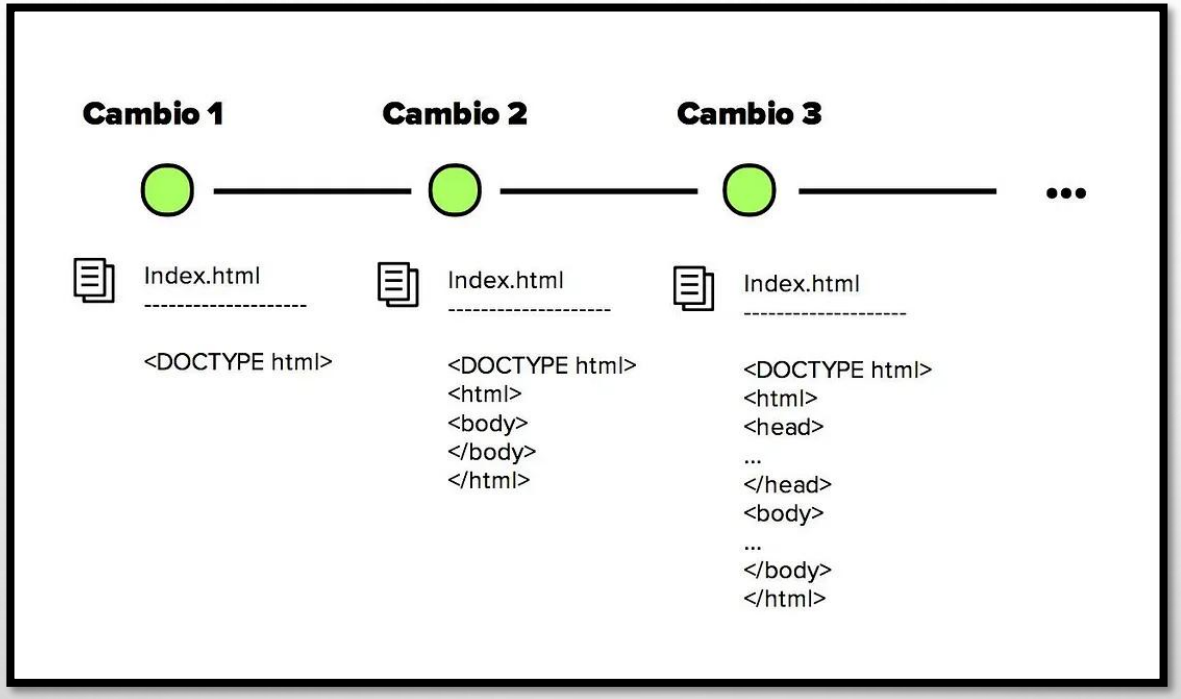


SISTEMA CONTROL DE VERSIONES



SISTEMA CONTROL DE VERSIONES

¿Que es control de versiones?



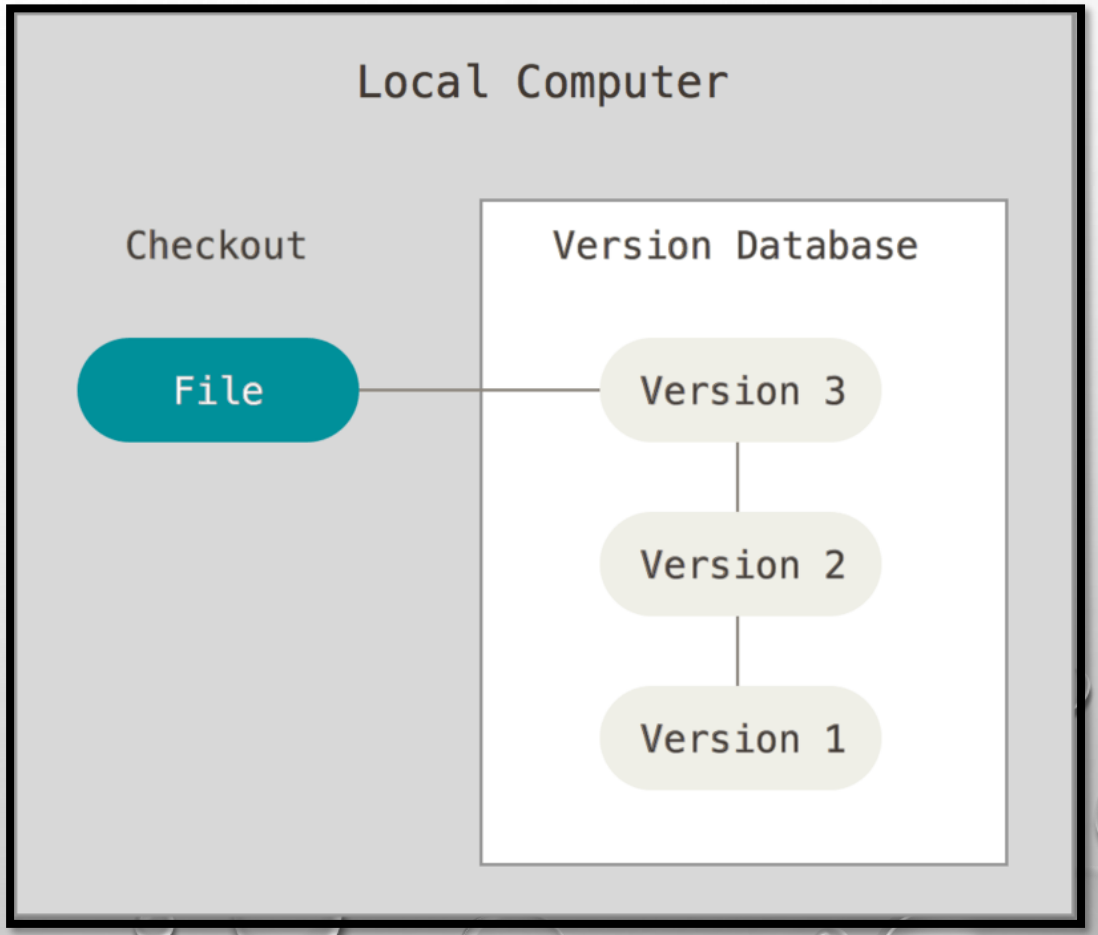
El control de versiones es un sistema que registra los cambios realizados sobre un archivo o conjunto de archivos a lo largo del tiempo de tal manera que sea posible recuperar versiones especificas más adelante.

SISTEMA CONTROL DE VERSIONES

Sistema control de versiones locales

Un método de control de versiones, usado por muchas personas, es copiar los archivos a otro directorio (quizás indicando la fecha y hora en que lo hicieron, si son ingeniosos). Este método es muy común porque es muy sencillo, pero también es tremendamente propenso a errores. Es fácil olvidar en qué directorio te encuentras y guardar accidentalmente en el archivo equivocado o sobrescribir archivos que no querías.

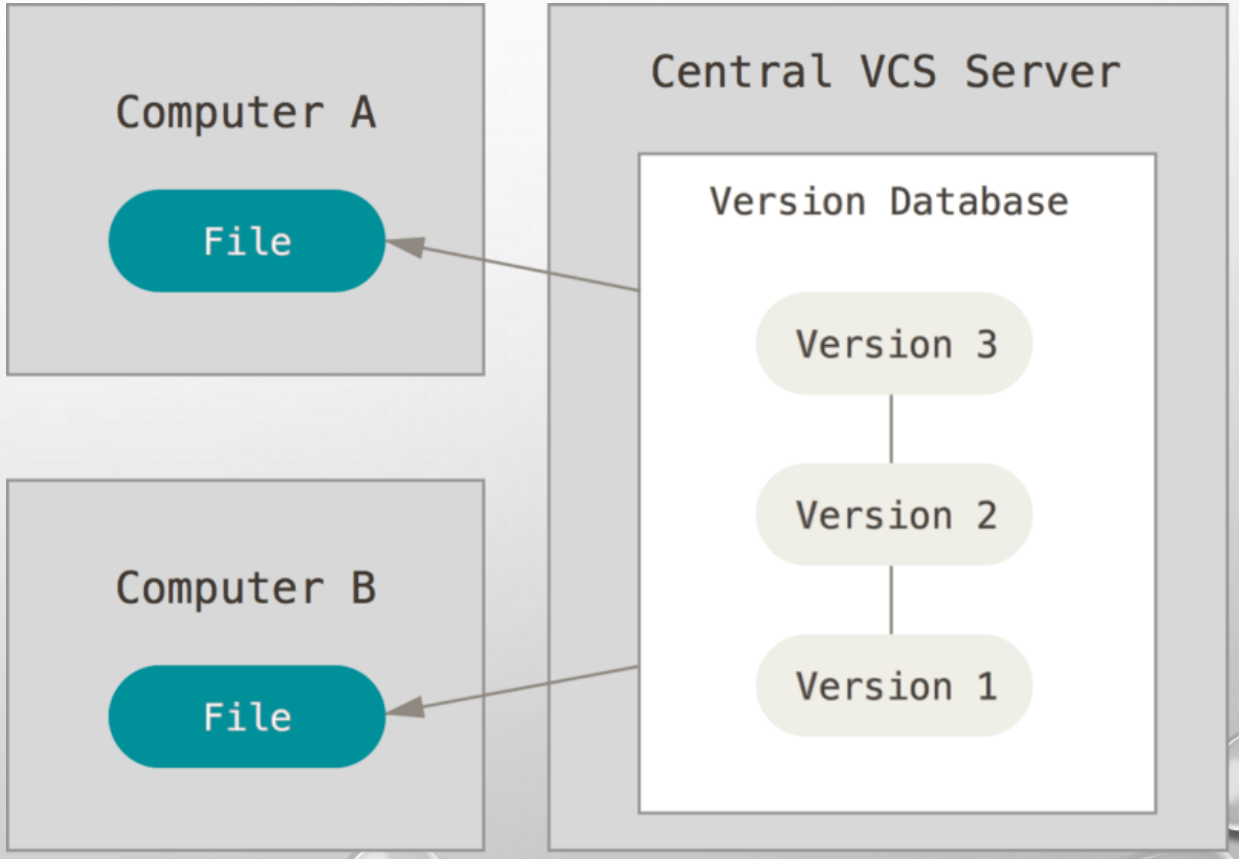
Para afrontar este problema los programadores desarrollaron hace tiempo VCS locales que contenían una simple base de datos, en la que se llevaba el registro de todos los cambios realizados a los archivos.



SISTEMA CONTROL DE VERSIONES

Sistemas de Control de Versiones Centralizados

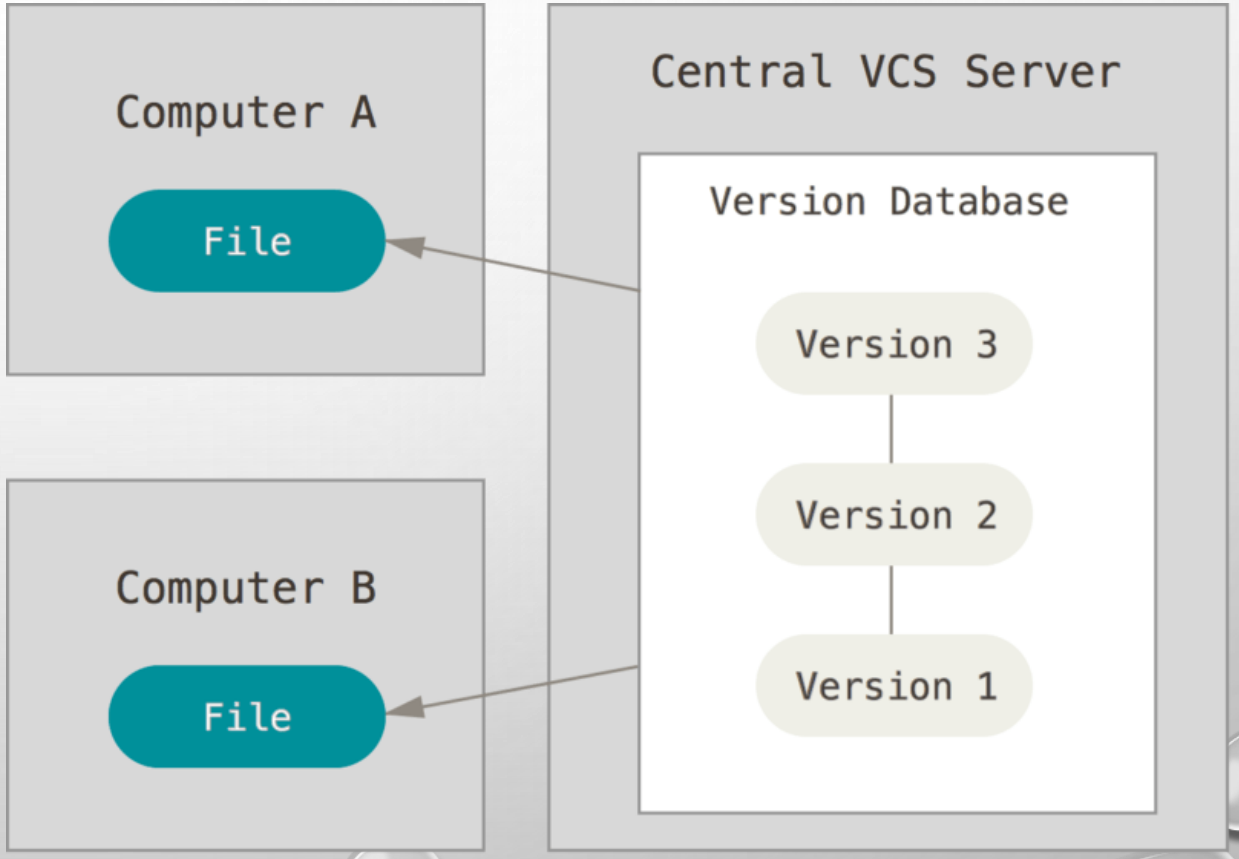
El siguiente gran problema con el que se encuentran las personas es que necesitan colaborar con desarrolladores en otros sistemas. Los sistemas de Control de Versiones Centralizados (**CVCS** por sus siglas en inglés) fueron desarrollados para solucionar este problema. Estos sistemas, como Subversion y Perforce, tienen un único servidor que contiene todos los archivos versionados y varios clientes que descargan los archivos desde ese lugar central. Este ha sido el estándar para el control de versiones por muchos años.



SISTEMA CONTROL DE VERSIONES

Sistemas de Control de Versiones Centralizados

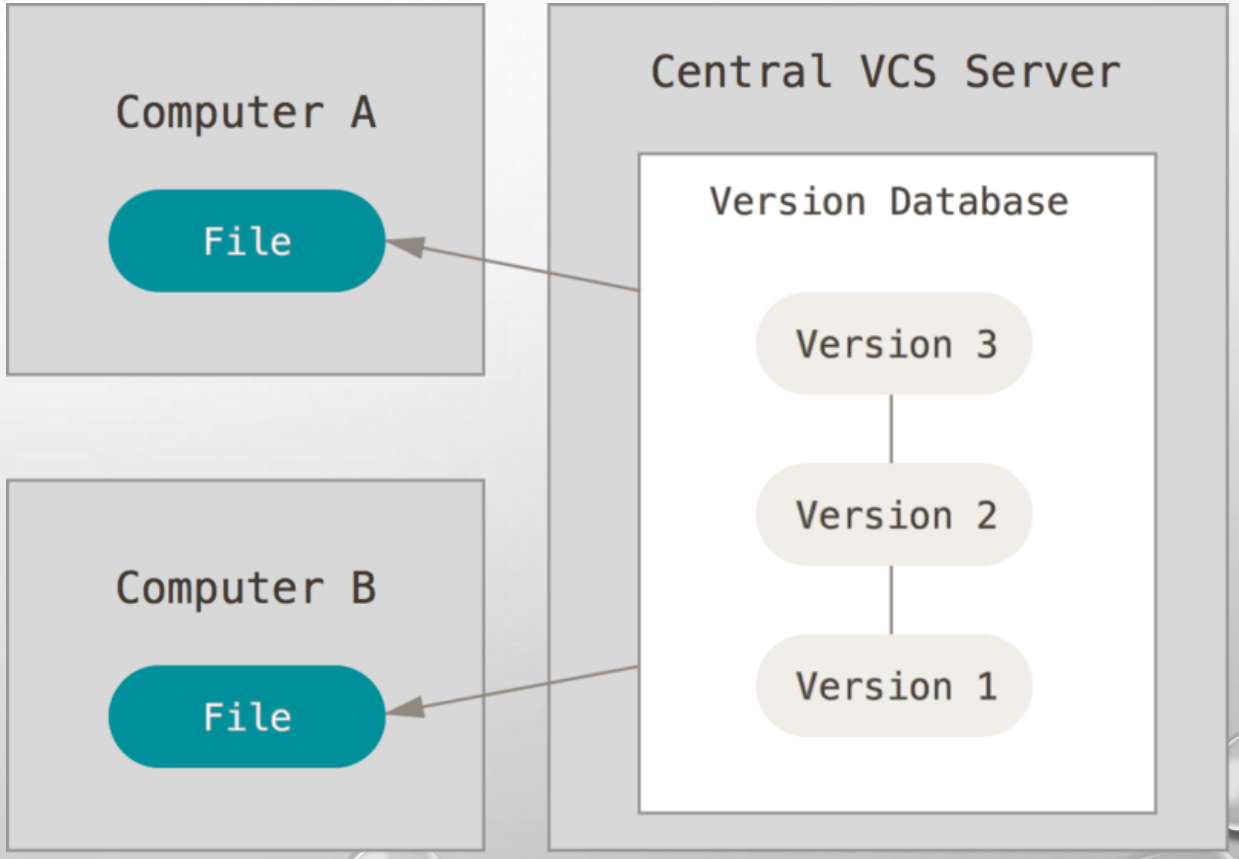
Esta configuración ofrece muchas ventajas, especialmente frente a VCS locales. Por ejemplo, todas las personas saben hasta cierto punto en qué están trabajando los otros colaboradores del proyecto. Los administradores tienen control detallado sobre qué puede hacer cada usuario, y es mucho más fácil administrar un CVCS que tener que lidiar con bases de datos locales en cada cliente.



SISTEMA CONTROL DE VERSIONES

Sistemas de Control de Versiones Centralizados

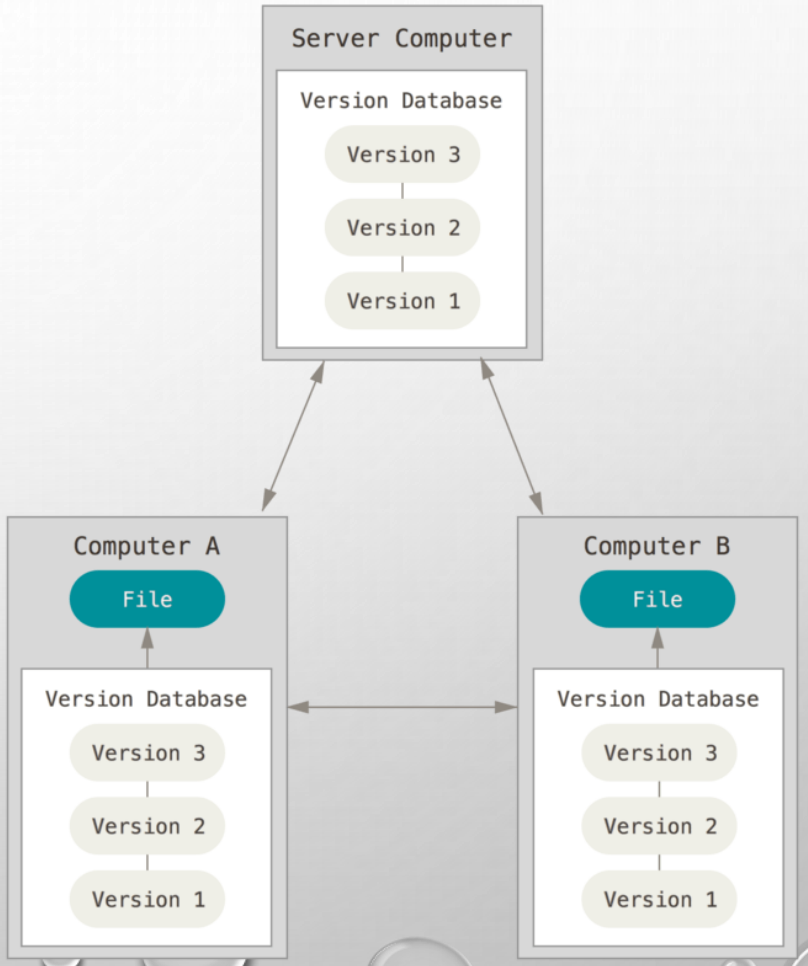
Sin embargo, esta configuración también tiene serias desventajas. **La más obvia es el punto único de fallo que representa el servidor centralizado**. Si ese servidor se cae durante una hora, entonces durante esa hora nadie podrá colaborar o guardar cambios en archivos en los que hayan estado trabajando. Si el disco duro en el que se encuentra la base de datos central se corrompe, y no se han realizado copias de seguridad adecuadamente, se perderá toda la información del proyecto, con excepción de las copias instantáneas que las personas tengan en sus máquinas locales. Los VCS locales sufren de este mismo problema: Cuando tienes toda la historia del proyecto en un mismo lugar, te arriesgas a perderlo todo.



SISTEMA CONTROL DE VERSIONES

Sistemas de Control de Versiones Distribuidos

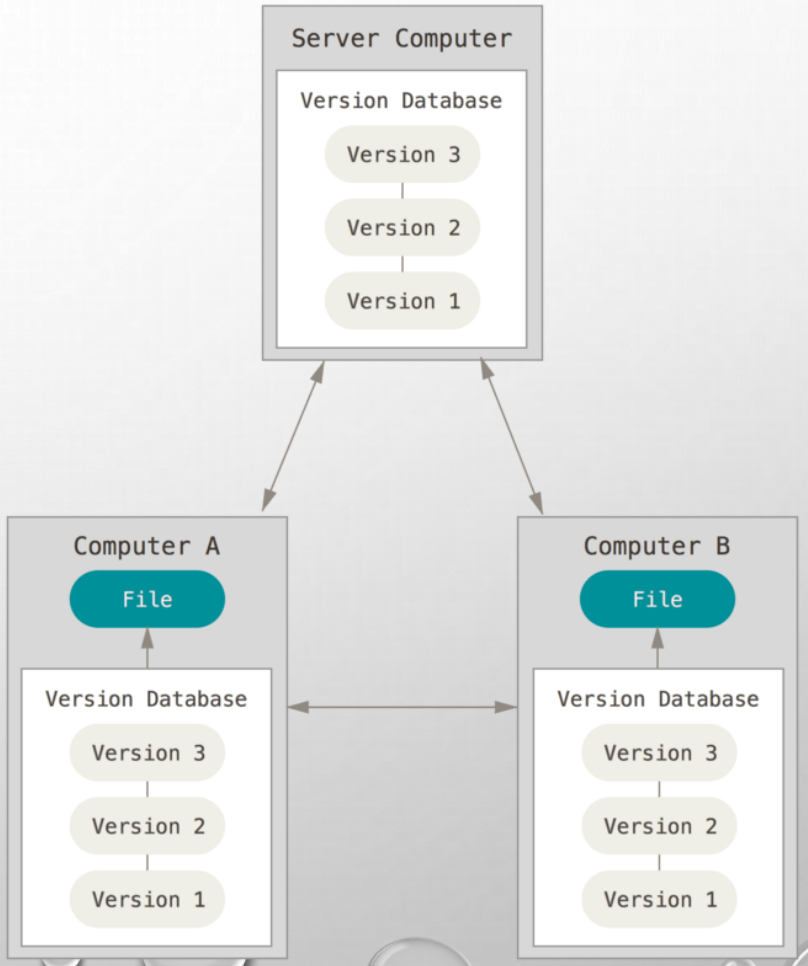
Los sistemas de Control de Versiones Distribuidos (DVCS por sus siglas en inglés) ofrecen soluciones para los problemas que han sido mencionados. En un DVCS (como Git, Mercurial, Bazaar o Darcs), los clientes no solo descargan la última copia instantánea de los archivos, sino que se replica completamente el repositorio. De esta manera, si un servidor deja de funcionar y estos sistemas estaban colaborando a través de él, cualquiera de los repositorios disponibles en los clientes puede ser copiado al servidor con el fin de restaurarlo. Cada clon es realmente una copia completa de todos los datos.



SISTEMA CONTROL DE VERSIONES

Sistemas de Control de Versiones Distribuidos

Además, muchos de estos sistemas se encargan de manejar numerosos repositorios remotos con los cuales pueden trabajar, de tal forma que puedes colaborar simultáneamente con diferentes grupos de personas en distintas maneras dentro del mismo proyecto. Esto permite establecer varios flujos de trabajo que no son posibles en sistemas centralizados



SISTEMA CONTROL DE VERSIONES

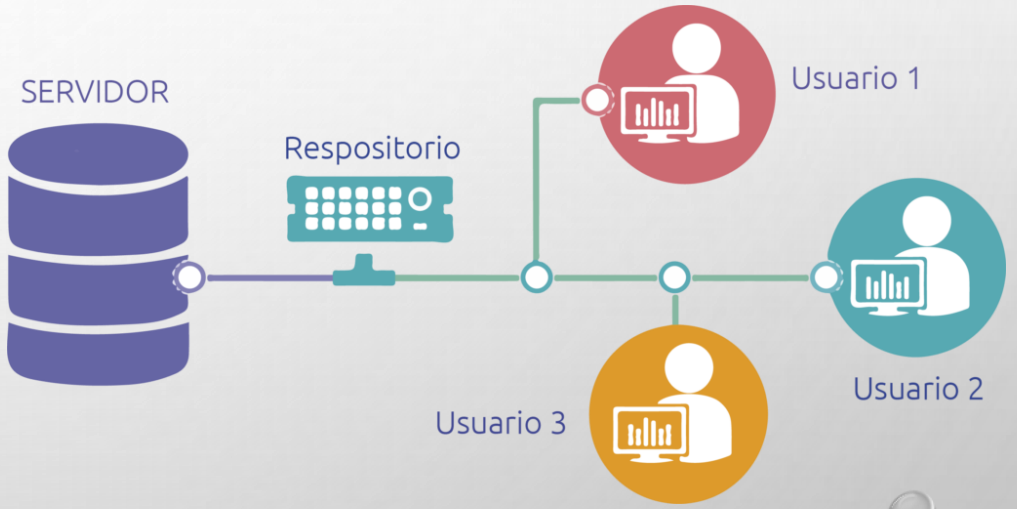
Ventajas y desventajas

Ventajas ✅

- Historial Completo: Puedes auditar cualquier cambio desde el día 1.
- Ramificación (Branching): Permite trabajar en una función nueva sin romper lo que ya funciona.
- Colaboración: Varias personas pueden tocar el mismo archivo y el sistema ayuda a "fusionar" (merge) sus cambios.
- Seguridad: Tienes respaldos en cada computadora del equipo.

Desventajas ❌

- Curva de aprendizaje: Git, por ejemplo, puede ser confuso al principio.
- Conflictos de fusión: Cuando dos personas cambian la misma línea, el sistema se detiene y pide intervención humana.
- Gestión de archivos binarios: No son ideales para fotos o videos muy pesados (aunque existen extensiones como LFS).



SISTEMA CONTROL DE VERSIONES

Curiosidades

Linus Torvalds y git.

Linus Torvalds creó Git en 2005 para gestionar el desarrollo del kernel de Linux tras el fin de la licencia gratuita de BitKeeper. Diseñado en solo 10 días, buscaba velocidad, integridad de datos y soporte distribuido para flujos de trabajo no lineales. Se convirtió en el estándar mundial de control de versiones

Git y github

- Git (Herramienta Local): Es el software instalado en tu máquina que registra cada cambio en tus archivos, permitiendo volver a versiones anteriores.
- GitHub (Plataforma en la Nube): Es el servicio web que utiliza Git, permitiendo subir repositorios locales para compartir, colaborar (pull requests) y gestionar tareas (issue tracking).

SISTEMA CONTROL DE VERSIONES

Conclusiones

El fin del miedo al error: Los VCS transforman el error humano de una "catástrofe" a una "lección recuperable". Nos da la libertad de fallar rápido y corregir más rápido.

Colaboración sin fronteras: Ya no importa si el equipo está en la misma oficina o en tres continentes distintos; el sistema garantiza que todos trabajen sobre la misma verdad técnica.

Auditoría perfecta: En un mundo regulado y profesional, saber quién hizo qué y por qué no es vigilancia, es responsabilidad y transparencia.

Estándar de la industria: No saber usar un VCS (específicamente Git) hoy en día es equivalente a no saber escribir código profesional. Es el lenguaje común del desarrollo.

Integración Continua (CI/CD): El control de versiones es el primer paso para la automatización. Sin un historial sólido, es imposible implementar procesos de despliegue automático o pruebas de calidad.

Documentación viva: El historial de commits es, en muchos casos, una mejor documentación que un manual en PDF, porque explica la evolución real del software.

GRACIAS